



Reinforcement Learning Trading Agents in a BSE Limit Order Book Environment

1 Introduction

Autonomous trading agents rely on market observations to decide when and how to trade. In a limit order book (LOB) market, these observations can be represented at different levels, from recent prices to best bid/ask quotes and deeper order book information. Because trading decisions depend on liquidity, spreads, depth and order flow as well as prices, the choice of state representation can strongly affect what the agent is able to perceive and learn [1, 2].

This project uses the Bristol Stock Exchange (BSE), an open-source LOB exchange simulator, as the underlying market environment [3, 4]. A reinforcement learning (RL) agent is trained in this environment because trading can be formulated as a sequential decision-making problem, where the agent observes a market state, chooses an action and receives a reward based on the resulting trading outcome [5].

The project therefore addresses the following research question: **how do different levels of market perception and state representation affect the behaviour and performance of an autonomous trading agent in a simulated financial market?**

To answer this question, the project compares several ways of representing the BSE market state for a trading agent. These representations range from simple price-only information, to best bid/ask quotes, raw LOB depth, handcrafted microstructure features and learned encoded LOB representations. This comparison is motivated by the fact that more detailed market information does not necessarily make the agent more effective: raw LOB states may contain richer information, but they are also higher-dimensional, noisier and harder for a learning algorithm to use efficiently. In contrast, refined or structured representations may help the agent focus on the most relevant aspects of the market more effectively.

A Proximal Policy Optimization (PPO) agent is trained to choose between holding, limit buy, limit sell, market buy and market sell actions [6]. The agent is evaluated using profitability, risk and transaction-cost-related measures. Overall, this project examines what kind of state design can best improve an intelligent trading agent's ability to perceive market conditions and act effectively in a simulated financial market.

2 Related Work

2.1 Limit Order Book Representation and Market Perception

The limit order book (LOB) is a standard representation of an order-driven financial market. It records outstanding buy and sell limit orders at different price levels, so the best bid and ask, bid–ask spread and visible depth provide a structured view of liquidity and trading pressure. Market microstructure research shows that trading mechanisms, liquidity, spreads, depth and order flow influence price formation [1, 2]. This makes the LOB a practical basis for representing the market information perceived by an automated trading agent.

Previous studies also show that LOB-based variables can describe short-term market conditions. Cont, Kukanov and Stoikov show that order flow imbalance and market depth help explain price impact and short-horizon price changes [7]. Kearns and Nevmyvaka emphasise feature engineering in high-frequency trading, where raw order-level data can be transformed into interpretable features such as spread, imbalance and execution-related costs [8]. More recent work learns representations directly from raw LOB data: DeepLOB extracts spatial and temporal patterns from LOB snapshots [9], while BDLOB extends this approach using Bayesian deep learning and uncertainty estimates [10].

Together, these studies motivate this project to investigate how different state designs, from price-based observations to LOB-based and feature-based representations, can better represent market information and support an agent’s perception of market conditions.

2.2 Agent-Based Market Simulation

Agent-based financial market simulation provides a controlled way to study trading behaviour that would be difficult to test in real exchanges. In a simulated market, the exchange mechanism, trader population and available information can be specified explicitly, allowing different agent designs to be compared under repeatable conditions.

The Bristol Stock Exchange (BSE) is an open-source simulation of a continuous double auction and limit-order-book exchange [3, 4]. Its matching engine allows trader orders to interact through the order book, so agent actions can affect subsequent market states rather than being evaluated only against a fixed historical price series. This makes BSE suitable for studying autonomous trading agents in a controlled LOB environment.

BSE also supports simple background trader populations such as ZIC and ZIP traders, building on earlier work on zero-intelligence and minimal-intelligence trading agents [11, 12]. These traders provide dynamic order flow for evaluating the learning agent.

2.3 Reinforcement Learning for Trading Agents

Reinforcement learning provides a natural framework for autonomous trading because trading can be formulated as a sequential decision-making problem: the agent observes a market state, selects an action, receives a reward and continues interacting with the environment [5]. This formulation is especially relevant to trading tasks where decisions must be made repeatedly under uncertainty and where current actions influence later outcomes.

Previous studies have applied reinforcement learning to trade execution and limit order trading. Nevmyvaka, Feng and Kearns show that execution decisions can be learned from market state information [13]. Later work extends this idea to execution timing and limit order placement, where agents must balance execution opportunity, transaction cost and price risk [14, 15, 16]. Simulation environments such as ABIDES and mbt-gym further show how LOB markets can be combined with modern RL methods for controlled experiments [17, 18].

3 Environment and Agent Design

3.1 Overview of Environment and Agent Design

The environment used in this project is built on top of the Bristol Stock Exchange (BSE). The original BSE system provides the core limit-order-book exchange mechanism, including order submission, cancellation, public LOB publication and trade matching. This project keeps the BSE matching engine as the underlying market mechanism.

This project uses Proximal Policy Optimization (PPO) as the fixed learning algorithm for the trading agent, so that the experiments focus on state representation rather than algorithm choice [6]. The PPO agent is connected to BSE through two wrapper layers, TradingEnv and BSEMarket, as shown in Figure 1.

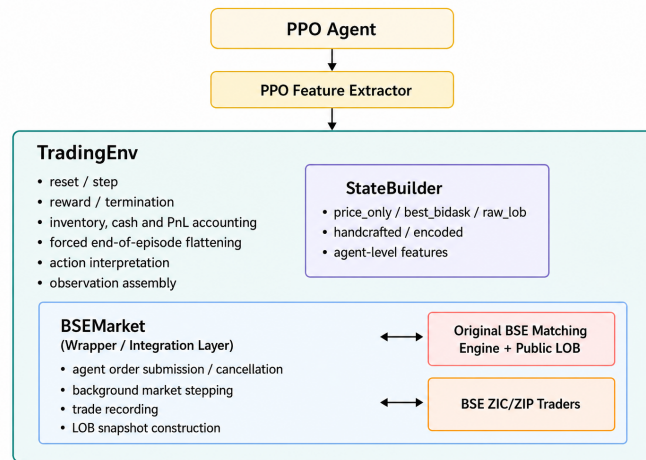


Figure 1: PPO agent and BSE environment structure

At the outer level, `TradingEnv` provides the Gymnasium-style reinforcement learning interface, including observation construction, action handling, reward calculation and episode termination. At the inner level, `BSEMarket` connects this interface to the original BSE matching engine. It creates the ZIC/ZIP background market, generates customer orders, advances the market through BSE, and converts the public order book into structured LOB snapshots. Through this two-layer structure, the PPO agent interacts only with `TradingEnv`, while `BSEMarket` handles communication with the underlying exchange mechanism.

The interaction between the agent and environment can be summarised as follows:

Algorithm 1 PPO trading-agent interaction loop

- 1: Observe the current LOB snapshot and agent portfolio state
 - 2: Build an observation vector using the selected state representation
 - 3: Select an action using the PPO policy
 - 4: Convert the selected action into an order submission or cancellation
 - 5: Execute the agent action in the BSE market
 - 6: Advance the background market using ZIC and ZIP traders
 - 7: Settle any trades involving the RL agent
 - 8: Compute PnL, reward, termination status, and the next observation
-

3.2 State Representation

State representation is the central design variable in this project. The environment supports five state types, ranging from minimal price information to raw and processed LOB-based representations. Table 1 summarises the market-level information included in each state type.

State type	Market feature dim.	Description
price_only	8	Uses basic price-based information, including the most recent trade or mid-price, lagged returns and short-term volatility. This represents the lowest level of market perception.
best_bidask	12	Extends price_only with top-of-book information, including best bid, best ask, bid–ask spread and mid-price. This gives the agent access to current liquidity and transaction-cost signals.
raw_lob	$4D + 1$	Uses bid and ask prices and quantities up to order book depth D , together with the most recent trade price. This gives the agent direct access to deeper LOB structure.
handcrafted	16	Uses engineered microstructure features, including spread, short-term returns, order imbalance, depth imbalance, micro-price deviation, volatility, top-of-book depth, total visible depth, best bid/ask prices, last trade price and recent trade direction.
encoded	encoder_latent	Represents the raw LOB vector using a lower-dimensional latent representation produced by a learned encoder. The encoder is pretrained offline on raw LOB snapshots collected from the background market before PPO training. The encoder variants are described below.

Table 1: Market-level state representations used by the PPO trading agent.

For the encoded representation, three encoder designs are considered:

- **MLP encoder:** maps the raw LOB vector into a latent vector through fully connected layers.
- **CNN encoder:** treats the LOB as a structured one-dimensional signal over price levels, allowing local relationships between neighbouring order book levels to be captured.
- **Autoencoder:** uses an encoder–decoder structure and learns a latent representation by reconstructing the original raw LOB input.

In addition to market-level information, every state representation includes three agent-level features, as shown in Table 2.

Feature	Purpose
Normalised inventory	The agent’s current position scaled by the maximum allowed inventory. This indicates whether the agent is long, short or close to its inventory limit.
Normalised PnL	The agent’s current profit or loss scaled relative to the initial cash level. This gives the policy information about the agent’s current trading performance within the episode.
Time remaining fraction	The proportion of the episode still remaining. This allows the agent to adjust its behaviour near the end of a session, especially when inventory needs to be managed before termination.

Table 2: Agent-level features appended to every state representation.

These agent-level features are included because the appropriate trading action depends not only on the observed market state, but also on the agent’s current portfolio condition. For example, a buy action may be reasonable when inventory is low, but less appropriate when the agent is already close to its maximum long position.

3.3 Action Space

The agent uses a discrete action space with five actions:

Action	Meaning
HOLD	Cancel the agent’s current resting order, if any, and submit no new order.
LIMIT_BUY	Submit a passive buy limit order at the current best bid, or one tick below the mid-price if no best bid is available. The order provides liquidity and may remain resting in the book until matched by a future sell order.
LIMIT_SELL	Submit a passive sell limit order at the current best ask, or one tick above the mid-price if no best ask is available. The order provides liquidity and may remain resting in the book until matched by a future buy order.
MARKET_BUY	Submit an aggressive buy order at the system maximum bid price so that it crosses the spread and attempts to execute immediately against the best available ask.
MARKET_SELL	Submit an aggressive sell order at the system minimum ask price so that it crosses the spread and attempts to execute immediately against the best available bid.

Table 3: Discrete action space of the RL trading agent.

This action design allows the agent to choose between waiting, providing liquidity through passive limit orders, or demanding liquidity through aggressive marketable orders. Buy and sell actions are constrained by the maximum inventory limit.

3.4 Reward Function

The reward function is based on the change in the agent’s portfolio value, with additional shaping terms used to discourage excessive inventory exposure, unnecessary transaction costs and inactive behaviour. The agent’s portfolio value is measured by its mark-to-market PnL:

$$\text{PnL}_t = \text{cash}_t + I_t \cdot m_t - \text{initial_cash}, \quad (1)$$

where I_t is the current inventory and m_t is the current mid-price.

The step reward is defined as:

$$R_t = \Delta \text{PnL}_t - \lambda_I |I_t| - C_t - \lambda_{\text{idle}} \mathbb{I}_{\text{idle},t} - \lambda_D D_t - \lambda_V V_t + \beta_{\text{trade}} \mathbb{I}_{\text{trade},t}. \quad (2)$$

where ΔPnL_t is the change in PnL, $\lambda_I |I_t|$ penalises inventory exposure, C_t is the transaction cost incurred during the step, and $\mathbb{I}_{\text{idle},t}$ indicates that the agent remains flat and does not trade during that step. The terms D_t and V_t represent drawdown and recent PnL volatility respectively, while $\mathbb{I}_{\text{trade},t}$ indicates whether a trade occurred. The small trade bonus encourages the agent to explore active trading behaviour rather than learning to remain permanently inactive.

This reward design encourages the agent to improve portfolio value while controlling inventory risk and trading costs. The same reward structure is used across all state representations to ensure a consistent training objective, although some experiments adjust the reward coefficients while keeping the functional form unchanged.

4 Experimental Setup and Implementation

4.1 Experimental Design

The experimental design is organised into three parts, as summarised in Table 4.

Experiment	Purpose	Compared methods
Exp. 1	Compare different levels of market information	price_only, best_bidask, raw_lob
Exp. 2	Compare different LOB representation methods	best_bidask, handcrafted, encoded/mlp, encoded/cnn, encoded/ae
Exp. 3 (Supp.)	Compare PPO agents with non-learning heuristic baselines	RL(best_bidask, handcrafted), random quoting, adaptive quoting, rule-based baseline

Table 4: Experimental matrix

Experiment 1: Information-level comparison

Experiment 1 compares the three information-level state representations: `price_only`, `best_bidask` and `raw_lob`. The aim is to examine whether giving the PPO agent access to richer market information improves trading performance under the same PPO algorithm, reward function, action space and evaluation protocol.

Experiment 2: LOB representation comparison

Experiment 2 examines whether processed LOB representations can help the RL agent use order book information more effectively. It compares `best_bidask` as a simple top-of-book reference, handcrafted as an engineered microstructure representation, and three learned encoded representations: `mlp`, `cnn` and `ae`. The purpose is to evaluate how different processing methods compare, and whether processed LOB information can improve PPO trading performance beyond the simpler best bid/ask representation.

Experiment 3: Supplementary policy-level validation

The supplementary experiment is not another state-representation ablation. Instead, it provides a policy-level validation in a denser and longer market regime with more active background order flow. It compares PPO agents using `best_bidask` and handcrafted states against three non-learning baselines: random quoting, ZIP-style adaptive quoting and a conservative rule-based strategy. These baselines respectively represent stochastic trading, simple adaptive response to short-term market signals such as spread and imbalance, and mean-reversion trading with inventory control. All methods are evaluated through the same `TradingEnv` interface using the same accounting rules, transaction-cost model, inventory limits and evaluation metrics.

This setting checks whether the handcrafted PPO agent remains competitive with `best_bidask` under more active order flow, and whether trained PPO policies outperform simple non-learning baselines.

4.2 Evaluation Metrics

The evaluation focuses on four main metrics:

- **Total PnL:** the final profit or loss achieved by the agent.
- **Sharpe ratio:** the risk-adjusted return calculated from PnL changes during an episode.
- **Maximum drawdown:** the largest decline from a previous PnL peak.
- **Transaction costs:** the cumulative trading costs incurred by the agent.

In addition, the implementation also records inventory variance and trade frequency as supplementary diagnostics to help interpret differences in agent behaviour.

Using multiple metrics allows the evaluation to capture profitability, risk exposure and trading efficiency. For Experiments 1 and 2, Welch’s t-test is used because the compared agents are trained independently and may have different performance variances. For Experiment 3, paired t-tests are used because policies are evaluated on the same shared evaluation seeds, so each comparison is made under matched simulated market conditions.

4.3 Implementation Details

The state representations, action space and reward function follow the design described in Section 3. The implementation was developed in Python using BSE for market simulation, a Gymnasium-style environment interface, Stable-Baselines3 for PPO, PyTorch for neural networks, and NumPy, Matplotlib and SciPy for data processing, plotting and statistical analysis. In the PPO implementation, each observation vector is passed through an MLP-based feature extractor before entering the policy and value networks, allowing state vectors with different original dimensions to be used within the same PPO framework.

Parameter	Value	Parameter	Value
Market environment		PPO training	
Market simulator	BSE	Learning rate	3×10^{-4}
Auction mechanism	CDA	Rollout steps	2048
Background buyers	5	Batch size	64
Background sellers	5	PPO epochs	10
Trader types	ZIC/ZIP	Discount γ	0.99
ZIC ratio	0.6	GAE λ	0.95
Price range	10–200	Clip range	0.2
Episode length	500	Entropy coeff.	0.01
LOB depth	5	Network	[128, 128]
Agent constraints		Reward shaping	
Initial cash	10000	Inventory penalty λ_I	0.015
Max inventory	± 10	Transaction cost rate	0.001
End flattening	True	No-trade penalty λ_{idle}	0.2
Experiment execution		Trade bonus β_{trade}	0.05
Independent runs	10	Encoded representation	
Training steps	200000	Encoders	MLP, CNN, autoencoder
Evaluation episodes	30	Encoder latent dim.	8

Table 5: Default implementation parameters defined in the main configuration file

Table 5 summarises the main implementation parameters. Some experiment scripts override

selected settings, but within each experiment all compared agents use the same reward definition, action space, environment dynamics and evaluation protocol.

4.4 Implementation and Experimental Challenges

Several implementation and experimental challenges were encountered. First, BSE was not originally designed as a reinforcement learning environment, so the original matching engine had to be wrapped in a Gymnasium-style interface while preserving its order book mechanics. The RL agent also had to be inserted as a special trader whose orders interact with the same matching engine and background ZIC/ZIP traders as all other market participants.

Second, the trading reward signal is noisy because short-term PnL depends on stochastic order flow, execution timing and inventory changes. Reward shaping with inventory penalties, transaction costs and trade-related terms was therefore used to discourage inactive behaviour and excessive inventory exposure.

Finally, the stochastic nature of the simulated market made the results sensitive to random seeds and episode-level order flow. To reduce the influence of individual lucky or unlucky episodes, each configuration was evaluated over multiple independent runs and repeated evaluation episodes. The results are therefore reported using mean and standard deviation, with statistical tests used in the results section to support the comparison between methods.

5 Results

This section presents the empirical results of the three experiments. Results are reported as mean $\pm$ standard deviation across independent runs, using the evaluation metrics and statistical testing procedures described in Section 4.2. Experiments 1 and 2 use 10 independent runs, while Experiment 3 uses 20 independent runs.

5.1 Experiment 1: Information-Level Comparison

Experiment 1 compares three levels of market perception: `price_only`, `best_bidask` and `raw_lob`. The purpose is to examine whether richer market information improves the performance of the PPO trading agent.

State type	Total PnL	Sharpe ratio	Max drawdown	Transaction Costs
<code>price_only</code>	-312.15 ± 110.11	-0.09 ± 0.03	1093.68 ± 364.69	2.05 ± 0.68
<code>best_bidask</code>	15.80 ± 268.08	0.01 ± 0.09	1024.21 ± 344.43	3.60 ± 1.87
<code>raw_lob</code>	-35.00 ± 443.13	0.01 ± 0.17	1131.59 ± 149.05	4.38 ± 3.33

Table 6: Experiment 1 results: information-level comparison.

The results show that `best_bidask` achieves the highest mean total PnL, followed by `raw_lob`, while `price_only` performs worst. The negative mean PnL of `price_only` suggests that price-only information is insufficient for stable profitable trading in this setting. By contrast, the positive mean PnL of `best_bidask` indicates that top-of-book information provides useful trading signals. Although `raw_lob` improves over `price_only` in mean PnL, it has the largest standard deviation, suggesting less stable learning performance.

Welch’s t-tests show that `best_bidask` significantly outperforms `price_only` in total PnL ($p = 0.0053$) and Sharpe ratio ($p = 0.0061$). However, the difference between `best_bidask` and `raw_lob` is not statistically significant in either total PnL ($p = 0.7726$) or Sharpe ratio ($p = 0.9880$). Similarly, the PnL difference between `raw_lob` and `price_only` is not statistically significant ($p = 0.0983$), despite the higher mean PnL of `raw_lob`. No pairwise comparison shows a statistically significant difference in maximum drawdown.

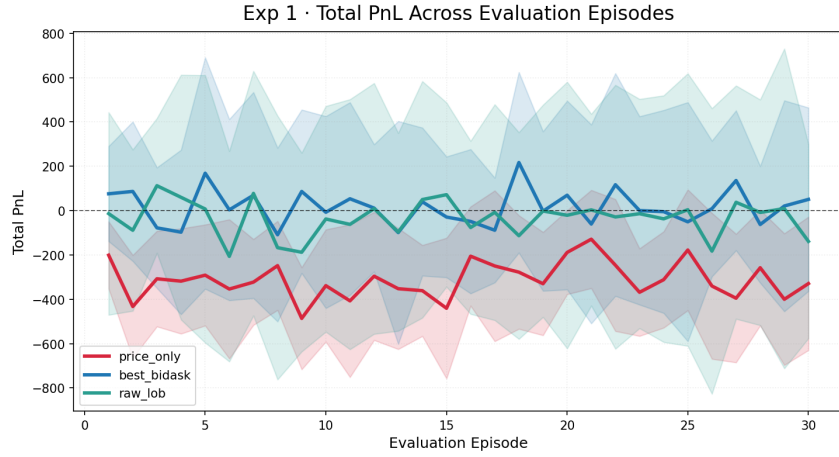


Figure 2: Total PnL across evaluation episodes in Experiment 1. Solid lines show mean performance across runs, while shaded regions indicate variability.

Figure 2 illustrates the same pattern visually. The `price_only` curve remains mostly below the other two representations, while `best_bidask` remains around zero PnL. The `raw_lob` representation shows larger variability across evaluation episodes.

Overall, Experiment 1 suggests that top-of-book information improves market perception, while raw LOB depth brings limited additional benefit over `best_bidask`.

5.2 Experiment 2: LOB Representation Comparison

Experiment 2 focuses on whether processed LOB representations can achieve better PPO trading performance than the simpler `best_bidask` baseline. Since Experiments 1 and 2 use the same market configuration, PPO hyperparameters, training budget and evaluation protocol, the `best_bidask` result from Experiment 1 is reused as the top-of-book baseline.

State type	Total PnL	Sharpe ratio	Max drawdown	Transaction Costs
handcrafted	270.02 ± 300.25	0.12 ± 0.14	1018.56 ± 147.99	7.79 ± 4.18
encoded/mlp	-201.67 ± 167.34	-0.06 ± 0.05	729.27 ± 595.51	1.37 ± 1.11
encoded/cnn	-339.09 ± 33.66	-0.10 ± 0.01	1216.44 ± 6.68	2.27 ± 0.01
encoded/ae	-241.18 ± 161.22	-0.07 ± 0.05	834.71 ± 548.35	1.59 ± 1.04
best_bidask	15.80 ± 268.08	0.01 ± 0.09	1024.21 ± 344.43	3.60 ± 1.87

Table 7: Experiment 2 results: LOB representation comparison.

The handcrafted representation achieves the best overall performance, with the highest mean total PnL and Sharpe ratio. All three encoded representations produce negative mean PnL, suggesting that the learned encoders used in this project do not extract sufficiently useful trading signals from raw LOB snapshots under the current setup.

Welch’s t-tests show that handcrafted significantly outperforms encoded/mlp ($p = 0.0010$), encoded/cnn ($p = 0.0002$) and encoded/ae ($p = 0.0005$) in total PnL. This provides strong evidence that the handcrafted microstructure representation is more effective than the simple learned encoder representations tested here.

The comparison with best_bidask is more nuanced: handcrafted has a much higher mean PnL (270.02 versus 15.80), but the difference is not statistically significant ($p = 0.0745$). This should therefore be interpreted as an indicative trend rather than definitive evidence that handcrafted outperforms best_bidask. Nevertheless, the higher mean PnL suggests that engineered microstructure features may have the potential to improve PPO trading performance.

The encoded representations also differ from each other. encoded/mlp significantly outperforms encoded/cnn in total PnL ($p = 0.0370$), although both remain negative on average. The encoded/cnn and encoded/ae representations are both significantly worse than best_bidask ($p = 0.0032$ and $p = 0.0265$, respectively).

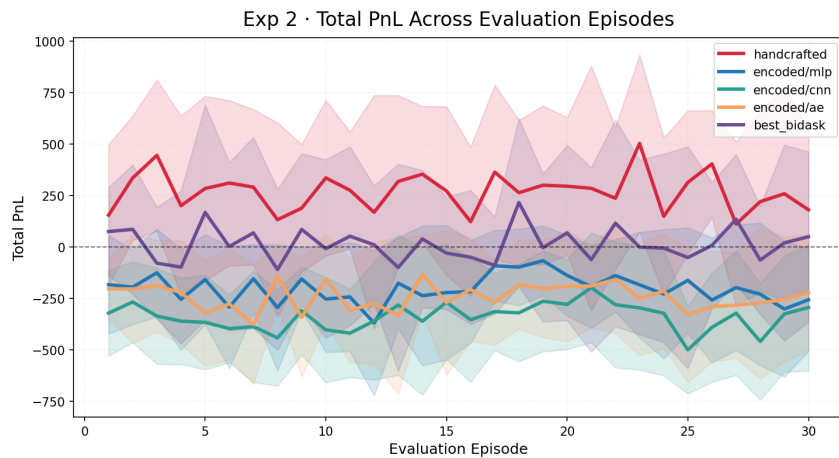


Figure 3: Total PnL across evaluation episodes in Experiment 2. Solid lines show mean performance across runs, while shaded regions indicate variability.

Figure 3 shows the same pattern visually. The handcrafted representation remains above the other representations for most evaluation episodes, while the encoded representations generally stay below zero PnL. The `best_bidask` baseline fluctuates around zero. Overall, Experiment 2 suggests that LOB information becomes more useful when it is processed into an appropriate state representation.

5.3 Experiment 3: Supplementary Policy-Level Validation

Experiment 3 provides a supplementary policy-level validation following Experiment 2. Since handcrafted achieved a higher mean PnL than `best_bidask` in Experiment 2 but the difference was not statistically significant, this experiment uses a denser and longer market setting to examine whether the two PPO representations can be more clearly distinguished. It also compares the trained PPO agents with non-learning heuristic baselines under the same `TradingEnv` execution and accounting framework.

Agent	Total PnL	Sharpe ratio	Max drawdown	Transaction Costs
RL (handcrafted)	762.40 $\pm$ 353.93	0.34 $\pm$ 0.22	777.64 $\pm$ 86.37	7.19 $\pm$ 2.49
RL (<code>best_bidask</code>)	455.70 $\pm$ 476.69	0.19 $\pm$ 0.22	861.88 $\pm$ 103.42	4.10 $\pm$ 2.72
Random quoting	-37.48 $\pm$ 20.99	-0.04 $\pm$ 0.02	518.90 $\pm$ 10.32	9.21 $\pm$ 0.05
Adaptive quoting	-90.31 $\pm$ 19.94	-0.09 $\pm$ 0.02	529.87 $\pm$ 11.31	7.23 $\pm$ 0.06
Rule-based	0.16 $\pm$ 21.85	0.00 $\pm$ 0.03	398.96 $\pm$ 11.12	11.37 $\pm$ 0.07

Table 8: Experiment 3 results: PPO policies versus non-learning baselines.

Both PPO agents outperform the three non-learning baselines in mean total PnL and Sharpe ratio. RL (handcrafted) performs best overall, followed by RL (`best_bidask`), while random quoting and adaptive quoting obtain negative mean PnL and the rule-based baseline remains close to zero. This indicates that the trained PPO policies learn more effective trading behaviour than fixed non-learning strategies under the same execution and accounting framework.

Paired t-tests further support this conclusion for total PnL. RL (handcrafted) outperforms random quoting, adaptive quoting and rule-based strategies with $p < 0.0001$ in all three comparisons. RL (`best_bidask`) also significantly outperforms these baselines, with p-values of 0.0003, 0.0001 and 0.0005, respectively.

The comparison between the two PPO agents is also significant in this supplementary setting. A direct paired t-test shows that RL (handcrafted) significantly outperforms RL (`best_bidask`) in total PnL, with $p = 0.0356$. This strengthens the observation from Experiment 2. While the handcrafted representation only showed a higher but non-significant mean PnL in Experiment 2, the denser and longer setting in Experiment 3 makes the performance difference clearer. The result suggests that engineered market-state features can provide more useful decision information than the simpler top-of-book representation when the agent is given more opportunity to

interact with the market.

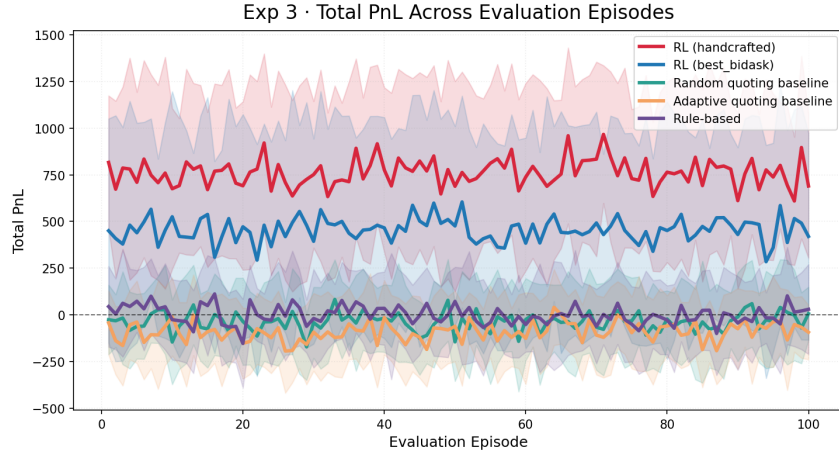


Figure 4: Total PnL across evaluation episodes in Experiment 3. Solid lines show mean performance across runs, while shaded regions indicate variability.

Figure 4 shows that the PPO curves remain mostly above zero PnL, whereas the heuristic baselines fluctuate around the zero line. The wider shaded regions for the PPO agents indicate greater variability across runs, consistent with their higher-return but more active trading behaviour.

Overall, Experiment 3 supports the usefulness of learning-based trading policies in the BSE environment. It also provides supplementary evidence that the handcrafted representation can support stronger learned behaviour than simpler top-of-book information.

6 Discussion

The results suggest that state representation plays an important role in how effectively a reinforcement-learning trading agent can use market information. The experiments do not support a simple “more information is always better” view. Instead, they indicate that effective selection and processing of market information can be more important than simply providing richer raw market data. In particular, top-of-book information improves over price-only observations because it gives the agent direct access to spread, liquidity and immediate execution-cost information, which are central to trading decisions in a limit order book market. However, directly adding raw LOB depth does not provide a clear additional advantage, suggesting that deeper order book information may be difficult for PPO to exploit when presented as a high-dimensional flat input.

The stronger performance of the handcrafted representation supports this interpretation. Handcrafted microstructure features encode market conditions such as spread, imbalance, depth, volatility and micro-price deviation in a compact and economically meaningful form. This reduces the burden on the policy network, because the agent does not need to discover these

structures directly from raw order book levels. Although Experiment 2 only shows an indicative advantage over `best_bidask`, the supplementary validation in Experiment 3 provides further evidence that engineered microstructure features can support stronger learned trading behaviour when the agent has more opportunity to interact with the market. By contrast, the learned encoded representations perform poorly, suggesting that simply compressing raw LOB snapshots may not preserve the microstructure signals most useful for PPO trading decisions.

The supplementary validation also shows that PPO agents can learn trading behaviour that is more effective than fixed non-learning heuristic rules under the same BSE-backed execution and accounting framework. However, profitability should not be interpreted in isolation. More profitable agents may trade more actively and incur higher transaction costs or larger drawdowns. Conversely, some heuristic baselines trade frequently without producing positive PnL. Total PnL, Sharpe ratio, maximum drawdown and transaction costs therefore need to be considered together, and the agent’s behaviour should be interpreted as a trade-off between profitability, risk exposure and trading efficiency rather than as a single-metric ranking.

7 Conclusion, Limitations and Future Work

This project investigated how market perception and state representation affect PPO trading agents in a BSE-based simulated limit order book market. The agent was trained under several state representations to choose between holding, passive limit orders and aggressive marketable orders.

The results show that top-of-book information improves performance over price-only information, while raw LOB depth is not automatically better, likely because it increases dimensionality, noise and learning difficulty. The strongest representation is the handcrafted microstructure state: it achieves the best mean performance in the LOB representation comparison, significantly outperforms the simple learned encoded representations, and also outperforms `best_bidask` in the supplementary denser and longer validation experiment. This supports the conclusion that carefully designed state features can help an RL trading agent use market information more effectively than raw or naively compressed order book input. Beyond the representation comparison, the supplementary experiment shows that learned PPO agents can outperform simple non-learning heuristic baselines under the same trading and accounting framework, although profitability should still be interpreted alongside risk and trading-cost measures.

There are several limitations. First, BSE is a simplified simulated market and does not include many features of real exchanges, such as latency, hidden liquidity, multiple assets or institutional order flow. The background market is also generated by relatively simple ZIC and ZIP traders, and the experiments are conducted under a small number of fixed BSE market configurations. Therefore, the learned policies may partially adapt to specific simulated market regimes, and the results should be interpreted as evidence within a controlled BSE setting rather than as direct

evidence of real-market profitability. Second, PPO training is stochastic and computationally expensive, so some comparisons may require more seeds or longer training to become more robust. This is especially relevant because the handcrafted representation shows a strong mean advantage over `best_bidask` in Experiment 2, but this difference only becomes statistically significant in the supplementary denser and longer validation setting. Finally, the action space is discrete and limited to five actions, whereas realistic trading systems may require continuous quote placement, variable order sizes and more flexible cancellation decisions.

Future work could extend the project in several directions. Learned LOB representations could be improved using auxiliary objectives such as short-term price prediction, execution-probability prediction or contrastive learning. These objectives may encourage the encoder to preserve decision-relevant information rather than only reconstructing the current order book snapshot. The encoder could also be trained together with the PPO policy, so that it learns to preserve information that improves trading rewards rather than only reconstructing the raw LOB input. Further experiments could test the agent under different market regimes, such as high-volatility or low-liquidity sessions, different ZIC/ZIP ratios, or more adaptive background trader populations. Finally, the action space could be extended to include quote distance, order size and cancellation timing, making the agent closer to a realistic trading system.

In summary, RL agents can learn useful trading behaviour in a BSE limit order book environment, but their performance depends strongly on how market information is represented. More information is not automatically better; it must be selected and structured to support effective learning and decision-making.

AI Statement

I confirm that this coursework submission represents my own work and understanding. AI tools, including ChatGPT, were used only for supportive purposes, such as proofreading, language refinement, debugging support and code-organisation suggestions. The project idea, design, implementation, experiments, results, analysis and conclusions were developed, checked and finalised by myself. AI tools were not used to generate experimental results or replace my own academic judgement.

References

- [1] Maureen O’Hara. *Market Microstructure Theory*. Blackwell, Cambridge, MA, 1995.
- [2] Larry Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, New York, 2003.
- [3] Dave Cliff. Bristol Stock Exchange: Open-source financial exchange simulator, 2012. URL <https://github.com/davecliff/BristolStockExchange>. Accessed: 2026-05-04.
- [4] Dave Cliff. BSE: A minimal simulation of a limit-order-book stock exchange. In *Proceedings of the 30th European Modeling and Simulation Symposium (EMSS 2018)*, pages 194–203, Budapest, Hungary, 2018. DIME University of Genoa.
- [5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [7] Rama Cont, Arseniy Kukanov, and Sasha Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jjfinec/nbt003.
- [8] Michael Kearns and Yuriy Nevmyvaka. Machine learning for market microstructure and high frequency trading. In David Easley, Marcos López de Prado, and Maureen O’Hara, editors, *High-Frequency Trading: New Realities for Traders, Markets and Regulators*, pages 91–124. Risk Books, London, 2013.
- [9] Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019. doi: 10.1109/TSP.2019.2907260.
- [10] Zihao Zhang, Stefan Zohren, and Stephen Roberts. BDLOB: Bayesian deep convolutional neural networks for limit order books. In *Third Workshop on Bayesian Deep Learning, NeurIPS 2018*, Montréal, Canada, 2018.
- [11] Dave Cliff and Janet Bruten. Minimal-intelligence agents for bargaining behaviors in market-based environments. Technical Report HPL-97-91, Hewlett-Packard Laboratories, Bristol, UK, 1997.
- [12] Marco LiCalzi and Paolo Pellizzari. Zero-intelligence trading without resampling. Working Paper 164, Department of Applied Mathematics, Ca’ Foscari University of Venice, 2008.

- [13] Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning*, ICML '06, pages 673–680, New York, NY, USA, 2006. ACM. doi: 10.1145/1143844.1143929.
- [14] Ciamac C. Moallemi and Muye Wang. A reinforcement learning approach to optimal execution, 2022. Working paper.
- [15] Matthias Schnaubelt. Deep reinforcement learning for the optimal placement of cryptocurrency limit orders. *European Journal of Operational Research*, 296(3):993–1006, 2022. doi: 10.1016/j.ejor.2021.04.050.
- [16] Avraam Tsantekidis, Nikolaos Passalis, and Anastasios Tefas. Modeling limit order trading with a continuous action policy for deep reinforcement learning. *Neural Networks*, 165: 506–515, 2023. doi: 10.1016/j.neunet.2023.05.051.
- [17] Michaël Karpe, Jin Fang, Zhongyao Ma, and Chen Wang. Multi-agent reinforcement learning in a realistic limit order book market simulation. In *Proceedings of the First ACM International Conference on AI in Finance*, ICAIF '20, New York, NY, USA, 2020. ACM. doi: 10.1145/3383455.3422570.
- [18] Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. Mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, ICAIF '23, pages 619–627, New York, NY, USA, 2023. ACM. doi: 10.1145/3604237.3626873.